

SEO Optimizer: a brief for the GUI designer

A Windows desktop app that crawls a website, tests it against 98 published SEO checks, and says whether the site is ready to launch. This page covers what a designer needs to know: the people, the objects, every state a screen must show, the rules the interface has to keep, and what exists today.

Version v0.1.1 (desktop, self-updating) Methodology corpus v5.0, reviewed 2026-09-11

Written 2026-09-28

What it is

SEO Optimizer is a **launch-readiness auditor**. You register a site, start an audit, and the engine crawls the site the way a search crawler would. Around 134 automated detectors read what it found, and the engine grades the result against a fixed methodology, the *corpus*. The output is a verdict, **GO** or **HOLD**, with every check explained and every explanation traceable to the pages it came from.

It is closer to a **compliance or QA tool** than a marketing dashboard. The audience is people who need to decide whether a site can ship and defend that decision later. Accuracy, traceability and honest "we don't know" states matter more than scores and charts.

98 checks in the current methodology (v5.0)	54 of them are launch gates that can block GO	8 lifecycle phases, from discovery to ongoing	134 detectors in 13 categories producing evidence
---	---	---	---

The 8 lifecycle phases

Every check belongs to one phase of a site's life. The phases are the natural top-level grouping for results. Each phase gets a percent-complete score.

PHASE	LABEL	CHECKS	EVIDENCE CLASS
0	Discovery & strategy	9	planning
1	Day 1 architecture	19	preflight

2	Pre-launch config	18	preflight
3	Content & trust	13	preflight
4	Pre-launch QA	12	preflight
5	Launch day	8	live
6	First 30 days	8	preflight
7	Ongoing	11	preflight

How much a machine can decide

Only 26 of the 98 checks can be passed by the machine alone. Most checks need a person to confirm, so the GUI is a **review workspace**, not only a report viewer.

AUTOMATION TIER	CHECKS	MEANING FOR THE UI
automated	26	The engine can pass or fail it on its own.
assisted	63	The engine gathers evidence and may fail it, but a person must confirm a pass.
attested	9	Governance nothing can observe. A person's signed statement is the only evidence.

Who uses it

The methodology names owner roles on every check: developer, SEO, content, legal, analytics, localization and more. In practice four jobs meet in the app. These are my reading of the model and the product owner should confirm them.

PERSON	COMES TO THE APP TO	NEEDS MOST
SEO lead / auditor	Set up sites, run audits, read results, work through the checks	A fast path from verdict to the failing checks, filters by phase, status and owner

Developer / fixer	Find out exactly what is wrong and where	Evidence down to the page URL and the raw observation; comparison with the previous audit to confirm a fix
Reviewer	Confirm assisted checks, attest governance checks, narrow scope with a rationale	A clear form, the evidence next to it, and when the attestation expires
Launch / decision owner	Decide whether to cut over and ship	One screen: GO or HOLD, what blocks it, and the READY FOR CUTOVER state

How an audit works

An audit runs in the background as a queued job. It can take minutes on a large site. It survives an app restart and is retried automatically if the engine (not the site) fails.

step 1 Submit Pick a site, a corpus version, optional crawl settings, release and supplied evidence.	step 2 Queue Waits for a slot. Only one crawl per host at a time.	step 3 Crawl Walks links and sitemaps within a page and depth budget, politely paced.	step 4 Probe Detectors read the crawl and record pass, fail, warn, n/a or error, per page or per site.	step 5 Grade Evidence is read against the corpus. Each check gets a state and readiness is frozen.	step 6 Review People confirm, attest or re-scope checks, then compare audits and decide.
---	--	--	---	---	---

Steps 1 to 5 are the engine. Step 6 is where people spend their time and where most of the design work is.

Once graded, an audit's verdict is **frozen**. A delivered report must not change later just because the scoring logic improved. Attestations recorded afterwards update the check states for that audit.

Objects & vocabulary

The words below are the product's own. Please use them in the interface as written, because they match the methodology workbook the users already know.

OBJECT	WHAT IT IS	KEY FIELDS A SCREEN SHOWS
--------	------------	---------------------------

Site	A website under audit, identified by its origin	name, origin (<code>https://www.example.com</code>), profile (core / extended), flags, AI crawler policy
Site flags	Short tags describing the site that bring conditional checks into scope. 32 are known, for example <code>ecommerce</code> , <code>multilingual</code> , <code>news</code> , <code>video</code> , <code>migration</code> , <code>faceted-nav</code> , <code>paywall</code>	A typo is refused before an audit runs, so this should be a picker, not free text
AI crawler policy	Which AI crawlers (GPTBot and others) the site wants to allow or block. A business decision, not something the crawl can observe	per agent: allow / disallow; approved by, approved at
Audit	One run against one site, pinned to one corpus version	status, created / started / finished, corpus version, decision, overall %, error
Check	One item of the methodology, e.g. <code>1.1</code> "Rendering strategy and raw/rendered parity"	id, phase, task, What to do, Done when, priority P0–P2, launch gate yes/no, owners, automation tier, sources
Check state	What this audit concluded about one check	applicability, status, coverage, evidence line, attestation expiry
Probe evidence	The detector observations behind a check state	detector id, outcome, summary, page URL, structured data
Attestation	A person's signed decision on a check, with an expiry date	attested by, status, applicability (+ rationale if "no"), statement, expires
Supplied evidence	Facts only a person can provide: Search Console exports, redirect map, staging URLs, CI results and 26 more sections	owner, recorded at, next review at. Missing owner or overdue review holds the check
Release	A named release of a site, e.g. <code>2026.10</code> , with its approvals and cutover record	scope revision, approver, decision owner, cutover authorizer and time

Review run	An append-only log entry: a person tested a gate in an environment on a date	check, environment, result, tested at, next review at. Never edited, only superseded
Launch readiness	First assessment: are all applicable launch gates passed?	GO / HOLD, gates outstanding, gates failed, scope decisions outstanding, lapsed attestations, violations
Cutover readiness	Second assessment, only when an audit names a release	READY FOR CUTOVER / HOLD, final GO / HOLD, blockers per gate, scope problems, launch-decision conflict

What a frozen verdict looks like

This is roughly the object behind a results screen (abridged, example values):

```
{
  "corpusVersion": "5.0",
  "readiness": {
    "decision": "HOLD",
    "gatesOutstanding": 17, "gatesFailed": 3,
    "applicabilityDecisionsOutstanding": 4,
    "attestationsLapsed": 1,
    "violations": [{ "checkId": "2.6", "kind": "skipped-applicable-gate", "message": "..." }]
  },
  "progress": [
    { "phase": 1, "active": 17, "notStarted": 4, "inProgress": 6,
      "passed": 5, "failed": 2, "skipped": 0, "percentComplete": 29, "scopeReview": 2 }
    // ... one row per phase
  ],
  "cutover": { "cutover": "HOLD", "final": "HOLD", "blockers": [ ... ], "launchDecision": "none" }
}
```

State reference

These are the complete value sets. Each needs a visual treatment, and the treatment should work without colour too (shape, icon or label), because several states differ only in meaning.

DIMENSION	VALUES	NOTES FOR DESIGN
Launch decision	GO HOLD	The headline of every audit. An ungraded audit has no decision yet, which is different from HOLD.

Cutover	READY FOR CUTOVER HOLD	Only when the audit names a release. Also a final GO / HOLD after launch-day checks.
Recorded launch decision	none agrees recorded conflict	conflict means someone recorded GO while the calculation says HOLD. It must be very visible.
Audit status	pending running complete failed cancelled	pending can also mean "waiting to retry", with the last error readable. Cancelled is a person's choice, failed is a problem to investigate. Never style them alike.
Check status	not-started in-progress passed failed skipped	A skipped launch gate is an integrity violation. The correct move is to set applicability to "no" with a rationale.
Applicability	yes no review	review is an open decision and blocks GO on a launch gate. no always carries a written rationale, so show it.
Coverage	verified attested unknown not-applicable	Where the result came from: machine, person, nobody, or out of scope. It is printed in every customer report.
Probe outcome	pass fail warn not-applicable error	error means the detector could not judge. It is not a finding about the site. Consider a treatment distinct from fail .
Priority	P0 P1 P2	Impact ranking only. It does not decide what blocks launch. The launch-gate flag does.
Review environment	planning pre-production production	Review runs also have the result reopened , for a change or incident logged against a result.

App update

checking

downloading 42%

ready, waits for idle

restarting

error

The app updates itself and waits until no audit is running. The user can choose "Restart now".

Rules the UI must keep

These come from the engine's design. Breaking one in the interface would mislead the person making a launch decision.

R1 Missing evidence is neither good nor bad news.

A check with no evidence is `not-started` / `unknown`. Never show it as green, and never as red.

R2 A machine may fail a check, but only an automated check may be passed by one.

An assisted check with clean evidence still waits for a person. The UI should show "ready for your confirmation", not "passed".

R3 A warning holds a check in progress.

The person confirming should see the warning itself, not a summary like "none failed".

R4 Attestations expire.

A lapsed attestation stays on the record but stops counting and holds its gate. Show expiry dates and flag the ones about to lapse.

R5 Excluding a check requires a reason.

Setting applicability to `no` needs a written rationale. Skipping an applicable launch gate is refused as a violation.

R6 Verdicts are frozen, and pinned to a corpus version.

Comparing two audits on different corpus versions needs a clear warning that verdicts may not be comparable.

R7 Cancelled is not failed.

Pages crawled before a cancel remain as evidence. A cancelled audit needs no investigation. A failed one does.

R8 The review log is append-only.

A correction is a new run with a later time. There is no edit or delete action on a review run.

R9 Supplied evidence needs an owner and a review date.

A record without an owner, or past its next-review date, holds its check. Show both on every record.

R10 Validation lists every problem at once, by field.

The API returns all problems together (`problems: ["origin: required", ...]`). Forms should show them inline next to each field.

The UI today

There is a working dashboard. It is a single page of plain HTML and JavaScript with no framework, shown inside the desktop window. It is functional and unstyled: system font, default form controls, one two-column layout. Treat it as a map of the data, not as a design to keep.

Sites list exists

left column

- Name and origin per site
- Click to open the site
- No way to add, edit or delete a site

Site detail exists

right column

- Readiness trend: a small line of overall % per audit, dots coloured GO / HOLD
- Audit history table: date, corpus, status, decision, overall %

Check results exists

below history

- Table of all checks: id, applicability, status, coverage, evidence line
- Shows ids only, not check titles
- No grouping by phase, no filters, no launch-gate marker

Evidence drill-down exists

on check click

- Each detector result: id, outcome, page URL, summary
- Structured data dumped as raw JSON

Attestation form exists

on check click

- Attested by, status, applicability, rationale (when "no"), statement, expiry date
- Success and error banners

Compare audits exists

below results

- Pick any site and audit as baseline
- Decision before → after, gates, per-phase % delta
- List of checks whose verdict changed; corpus mismatch warning

Version & update badge exists

fixed, bottom right

- Current version, update download progress, "Restart now"

Missing from the UI today

The engine and API already support most of these. The last group needs backend work before a screen can use it.

API ready, no screen yet

- Create, edit and delete a site, including flags, profile and AI crawler policy
- Start an audit, with crawl settings (max pages, depth, delay, robots, sitemaps, headless render)
- Attach supplied evidence to an audit (30 sections, currently YAML)
- Import a release and its review runs (currently a YAML file and CLI)
- Cutover readiness: blockers per gate, scope problems, final GO
- Integrity violations and lapsed attestations

Needs backend work first

- Check titles and text: the API does not serve the corpus yet (needs e.g. `GET /corpus/:version`)
- Cancel an audit: the engine supports it, but there is no endpoint
- Live crawl progress: today only status and queue state are available, not pages crawled so far
- List releases and review runs: import only, no read endpoint
- Known site flags for a picker: computed from the corpus, not exposed
- Database settings and "database unreachable" handling in the app

Journeys to design

In rough order of how often they happen, most frequent last.

1 First run.

The app needs a Postgres database, local by default. If it is unreachable, the user needs a clear explanation and somewhere to fix the connection, instead of an empty list.

2 Add a site.

Name, origin, profile. Optionally flags from a known list and an AI crawler policy. Warn that flags narrow scope: a missing flag excludes checks.

3 Start an audit.

Sensible defaults with an "advanced" area for crawl settings. Optionally name a release and attach supplied evidence. The audit id comes back immediately and the audit runs in the background.

4 Watch it run.

Queued (possibly waiting for another crawl of the same host), running, retrying after an engine error, complete, failed, cancelled. The app may be restarted mid-audit and resumes it.

5 Read the verdict.

GO / HOLD first, then why: failed gates, open scope decisions, lapsed attestations, violations. Then progress per phase.

6 Work the checks.

Filter by phase, status, launch gate, priority, owner role or automation tier. Open a check to see its requirement, its "Done when" test, the evidence and the pages involved.

7 Confirm, attest or re-scope.

Confirm assisted checks, attest governance checks, or set a check out of scope with a rationale. Always with an expiry.

8 Compare.

Against the previous audit of the same site to confirm fixes, or against another site. Show what changed and flag corpus mismatches.

9

Prepare cutover.

For a named release: the release record, the review-run log per gate, blockers, and the READY FOR CUTOVER state that the launch owner acts on.

API surface

The GUI talks to a local JSON API. The dashboard reaches it through `/api/*`. All endpoints exist today.

ENDPOINT	DOES	FEEDS
GET <code>/sites</code>	List sites	Sites list
POST <code>/sites</code>	Create a site	Add site
PATCH <code>/sites/:id</code>	Update any site fields	Site settings
DELETE <code>/sites/:id</code>	Delete a site and everything under it	Site settings (needs an in-page confirm step)
GET <code>/sites/:id/audits</code>	Audit history, newest first, with frozen readiness	Site overview, trend
POST <code>/audits</code>	Submit an audit, returns its id (202)	Start audit
GET <code>/audits/:id</code>	Audit row plus live queue state and attempt number	Audit status / progress
GET <code>/audits/:id/readiness</code>	Decision, per-phase progress, cutover	Verdict header
GET <code>/audits/:id/result</code>	Readiness plus every check state	Checks list
GET <code>/audits/:id/checks/:checkId/evidence</code>	Detector results behind one check, with page URLs	Check detail
POST <code>/audits/:id/attestations</code>	Record a person's decision on a check	Attest form

POST /releases (?dryRun=true)	Import a release and review runs; dry run validates only	Release editor
GET /version	App version	Version badge

Platform

- **Windows desktop app** (Electron), one window, installed per user. The window hosts the web UI, so the GUI is built with web technology and can also run in a browser during development.
- **Self-updating** from GitHub Releases. The version and update state need a permanent, unobtrusive place in the layout.
- **Data-dense.** 98 checks per audit, many pages of evidence per check, and structured detector data. Plan for tables, filters and a detail pane rather than cards.
- **Light and dark.** The current dashboard follows the system theme. Keep that.
- **No build step today.** The current UI is plain JavaScript. A dedicated GUI may bring a framework and a component library; that is an open decision.
- **Not code-signed yet.** Windows SmartScreen warns on first install. Onboarding copy may need to address that.

Open questions

Decisions for the product owner before or during design:

- Is this a single-user desktop tool, or will several people (auditor, reviewer, launch owner) use it on shared data? That decides whether attestations need a signed-in identity instead of a typed name.
- Is there a customer-facing report, such as a PDF or a shareable page, or is the app itself the report?
- Should supplied evidence and releases get proper forms, or file upload of the existing YAML formats, or both?
- How much of the methodology text (What to do, Done when, sources) should appear inside the app versus link out?
- Brand: the product has an app icon but no visual identity yet. Name, palette and type are open.

Compiled from the seo-optimizer repository (API, dashboard, core types and corpus v5.0) on 2026-09-28. Counts are from the v5.0 corpus files.